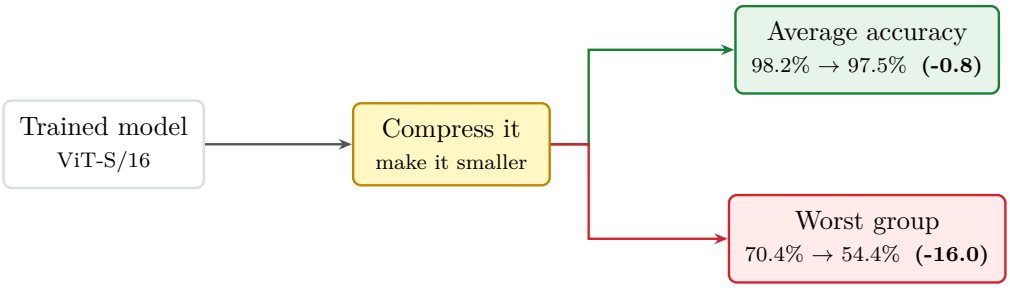
# Understanding this project

Compression vs. group robustness on Waterbirds

A plain-English tour of the idea and every file in the repo




**21× worse**, and the average almost hides it

Everything in here was checked against the actual files in the repo.
Numbers come from `results/sweep.csv` and `results/summary_table.csv`.

# Contents


**1 The 60-second version** **3**

**2 The concepts, with zero code** **3**
- 2.1 The dataset: Waterbirds . . . 3
- 2.2 What the model actually learns . . . 4
- 2.3 Why average accuracy lies . . . 4
- 2.4 What "compression" means here . . . 5
- 2.5 The one rule that makes this a clean experiment . . . 5

**3 The pipeline, end to end** **5**
- 3.1 What a ViT block looks like, and where we cut . . . 6

**4 Every file, one by one** **7**
- 4.1 `configs/base.yaml` – every number lives here . . . 7
- 4.2 `src/metrics.py` – what "group" and "worst group" mean . . . 7
- 4.3 `src/data.py` – loading images, fast . . . 8
- 4.4 `src/model.py` – building the model and freezing the scope . . . 8
- 4.5 `src/evaluate.py` – one measurement pass . . . 8
- 4.6 `src/train.py` – fine-tuning, deliberately naive . . . 9
- 4.7 `src/compress/` – the four methods, one contract . . . 9
- 4.8 `src/sweep.py` – the thing that actually runs the experiment . . . 12
- 4.9 `src/figures.py` – turning the CSV into pictures . . . 12
- 4.10 Phase 2 files (written, not run yet) . . . 12
- 4.11 `tests/` – the safety net . . . 13

**5 The tricky bits** **13**
- 5.1 Collapse: when the gap narrows and it means nothing . . . 13
- 5.2 Head removal isn't monotonic, and that's fine . . . 14
- 5.3 Why the adjusted average, one more time . . . 14
- 5.4 The resume bug (a real one we found and fixed) . . . 14

**6 Phase 2: can we fix it, and does the fix survive?** **14**
- 6.1 The fix: throw away the hardest training images . . . 15
- 6.2 The measurement: retention ratio . . . 15
- 6.3 Two things the code refuses to do . . . 16
- 6.4 One non-obvious engineering bit . . . 16

**7 How to actually read this code** **16**
- 7.1 A reading order that works . . . 16
- 7.2 The exercise that makes it stick . . . 16
- 7.3 Commands you'll actually use . . . 17
- 7.4 The three things to remember . . . 17

# 1. The 60-second version

Here's the whole project in one breath:

> **The question**
> You train a model. It works. Then you shrink it so it's cheaper to run – everyone does this before shipping. **Does shrinking it quietly break the groups it was already bad at?**

And the answer we got: **yes, and if you only look at average accuracy you'd never notice.**

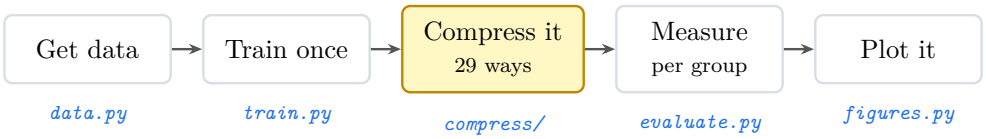
Chop 30% of the weights out of our model and average accuracy drops by **0.8 points**. Sounds free. But the worst-performing group of images drops by **16 points**. That's 21 times bigger. Report only the average and you'd call that compression free, ship it, and the model would be meaningfully worse for exactly the cases it already struggled with.

It gets sharper. Delete 6 attention heads out of 72 – that's 2.8% of the weights we're allowed to touch – and the average loses 1.7 points while the worst group loses **22.3**.

### The three words you need

- **Shortcut.** Our dataset is birds. Water birds are usually photographed on water, land birds on land. So the model learns "water background = water bird" instead of actually learning what a bird looks like. It's lazy, and it works most of the time.
- **Group.** Split the test set by (what bird it is) × (what background it's on). Four groups. Two of them agree with the shortcut, two fight it. The ones that fight it are where the model is weak.
- **Worst-group accuracy.** Accuracy on the single worst of those four groups. This is the number that actually tells you whether the model learned birds or learned backgrounds.

### What the repo does, start to finish




That's Phase 1, and it's done. Phase 2 (code written, not run yet) asks the follow-up: if you *fix* the robustness problem first, does compression undo the fix? More on that in Section 6.

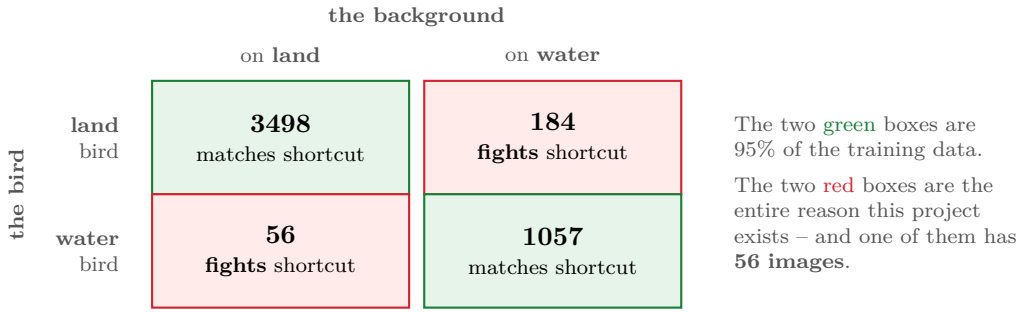
# 2. The concepts, with zero code

Skip this part if you already get shortcut learning. Otherwise this is the foundation for everything else.

## 2.1 The dataset: Waterbirds

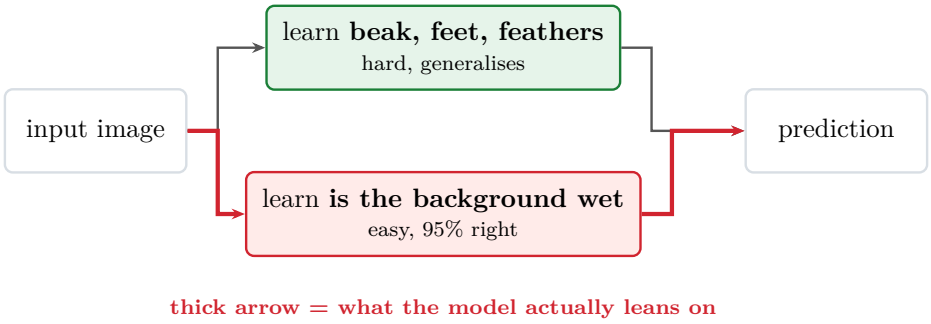
Waterbirds is a dataset built on purpose to be a trap. Someone took photos of birds, cut the birds out, and pasted them onto new backgrounds. They controlled exactly how often each bird type lands on each background type.

Here's what the training set looks like. Each box is a group, and the number is how many training images are in it:




Look at that imbalance. 3498 land birds on land, but only 56 water birds on land. If you're a model trying to minimise your loss, you will notice that "is the background wet?" predicts the label correctly 95% of the time, and that's a much easier thing to learn than bird anatomy.

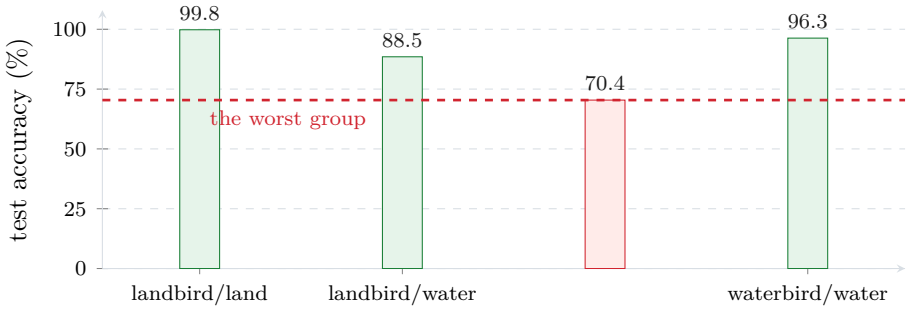
## 2.2 What the model actually learns




Nothing in plain training punishes the shortcut. It's not a bug in the model, it's the rational move given the data. That's what makes it interesting.

## 2.3 Why average accuracy lies

Average accuracy lies to you. Our model scores 98.2% and you'd think you're done. But break it down by group:




There it is. **99.8%** on the easy group, **70.4%** on water birds photographed on land – the group with 56 training images that fights the shortcut. A 29-point spread, and the average buries it.

So we track three numbers, and you need to know the difference:

| Number | On our model | What it means |
|---|---|---|
| plain average | 91.7% | straight mean over the test set |
| adjusted average | 98.2% | reweighted to match *training* group frequencies |
| **worst group** | **70.4%** | the single weakest group. the honest one |

**Why two different "averages"?**

The test set is way more balanced than the training set (test is 2255/2255/642/642; training is 3498/184/56/1057). So a plain test average over-weights the rare groups compared to how often they actually show up in training. The *adjusted* average reweights per-group accuracy by training frequency, and that's the ∼98% number everyone in this research area quotes. We report it so our numbers are comparable to published work. The code is `metrics.py`, function `adjusted_average`.

### 2.4 What "compression" means here

Compression means making the model smaller or cheaper after it's already trained. We try four flavours, and they're genuinely different operations:

**Magnitude pruning**
set the smallest individual weights to zero
*unstructured* – scattered weights

**Head removal**
delete whole attention heads
*structured* – whole chunks

**FFN neuron removal**
delete whole neurons from the feed-forward layers
*structured* – whole chunks

**Quantization**
store weights with fewer bits (8, 6, 4, 3)
not removal – just *less precision*

Each one has a knob we turn. Prune 30%, 50%, 70%... remove 6 heads, 12 heads, 24... quantize to 8 bits, 6, 4, 3. 29 settings total. For every single one we re-measure all four groups.

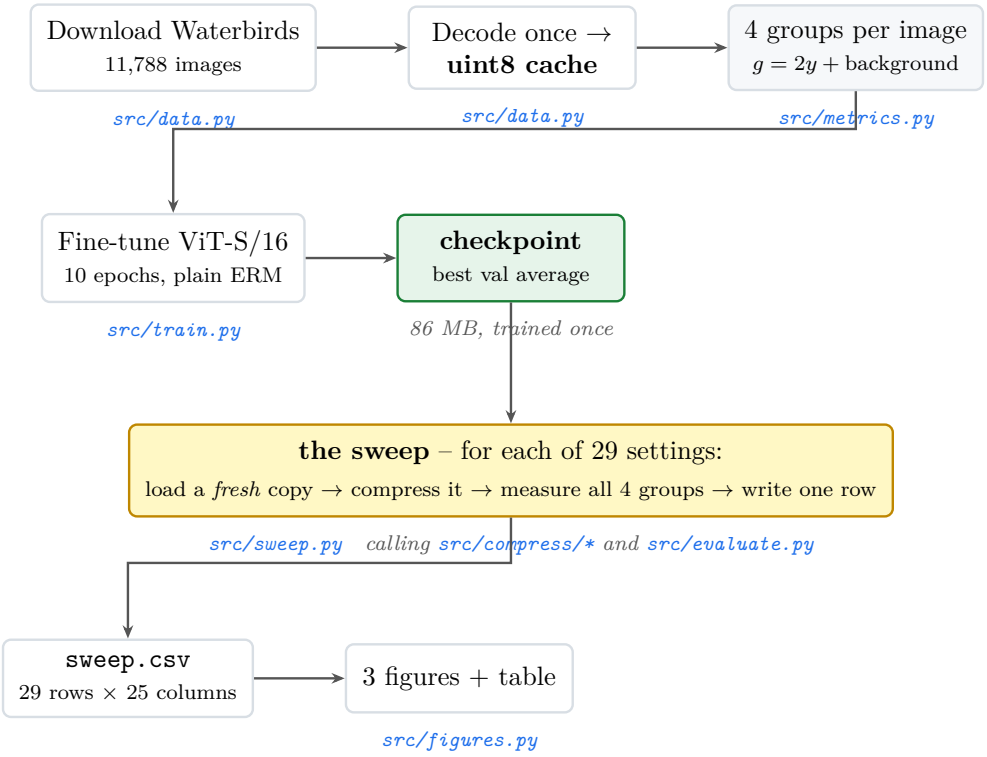
### 2.5 The one rule that makes this a clean experiment

**We never retrain after compressing**

This is the whole design. Compress the finished model, measure immediately, done. If we fine-tuned afterwards the model would just re-learn the shortcut and we'd be answering a completely different question. Same reason training is deliberately *group-blind*: no group labels, no reweighting, no balancing. The model is *supposed* to learn the shortcut. If we fixed that during training, worst-group accuracy would start high and there'd be nothing to measure.

## 3. The pipeline, end to end

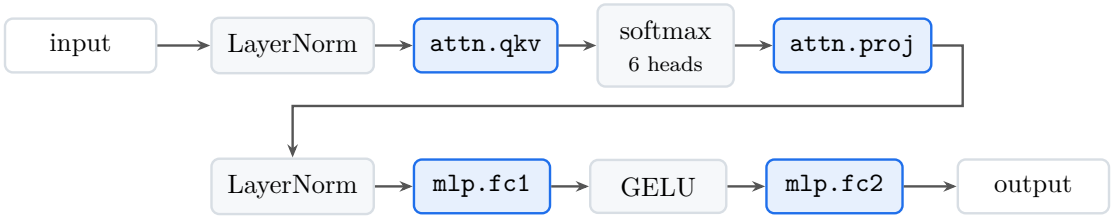
Now the code. Before looking at any single file, get this picture in your head – everything else is a detail hanging off it.




Read that top to bottom and you've got the shape of the repo. Now the important bit about *where* in the model the compression actually happens.

### 3.1 What a ViT block looks like, and where we cut

Our model is ViT-S/16: 12 identical blocks stacked on top of each other. Each block has two halves – an attention part and a feed-forward (MLP) part. Here's one block, with the four weight matrices we're allowed to touch highlighted:



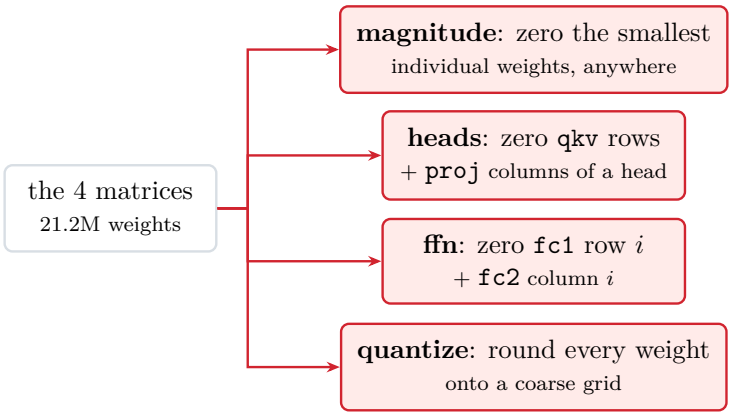

**The four blue boxes are the only things any compression method may touch.** 12 blocks × 4 matrices = **21,233,664 weights**.

**The "scope" and why it's frozen**

Everything else – the patch embedding, the position embeddings, the class token, every LayerNorm, every bias, and the final classifier head – is **off limits**, for every method, always. That's the list in `model.py` called `TARGET_SUFFIXES`.

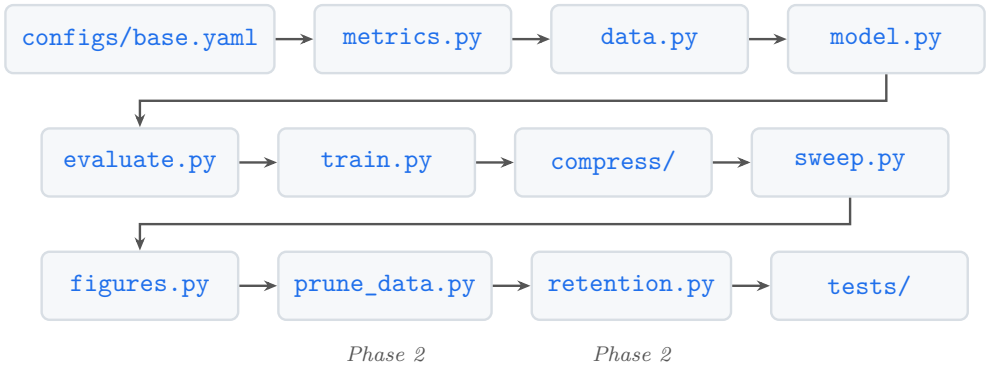
Why bother? Because "we pruned 30%" has to mean the same thing in all four experiments. If head removal were allowed to touch LayerNorms and magnitude pruning wasn't, the sparsity numbers wouldn't be comparable and the whole results table would be meaningless. One scope, decided once, reused everywhere.

And here's where each method attacks that scope:




## 4. Every file, one by one

Read them in this order – each one only depends on the ones above it.




### 4.1 `configs/base.yaml` – every number lives here

**What it does:** holds every constant the pipeline depends on. Learning rate, epochs, which sparsity levels to sweep, where files go.

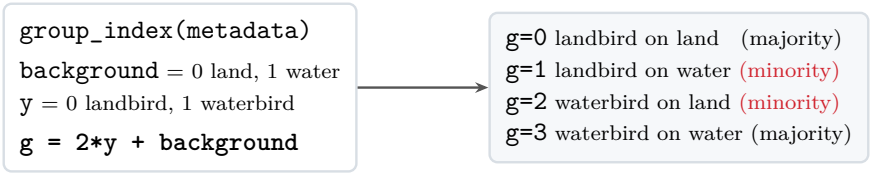
**Why it exists:** so no number is hardcoded in two places. If you want to sweep different sparsity levels, you edit this file and nothing else.

**Look at:** the `sweep:` block – those lists are literally the 29 experiments. `magnitude_sparsity` has 7 values, `heads_removed` 7, `ffn_fraction` 7, and quantize is 4 bit-widths $\times$ 2 granularities = 8. That's $7 + 7 + 7 + 8 = 29$.

### 4.2 `src/metrics.py` – what "group" and "worst group" mean

**What it does:** turns predictions into group-aware accuracy numbers. No model code, no PyTorch layers – just counting.

**The one function to understand:**



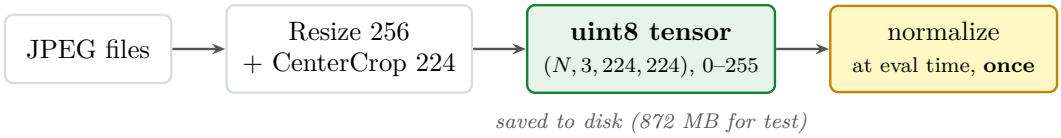
**The tricky bit:** `adjusted_average`. It does *not* compute a plain mean. It takes per-group accuracy and reweights by how common each group was **in training** (72.9%, 3.8%, 1.2%, 22.0%). That's why our headline is 98.2% and not 91.7%. Both are correct; they answer different questions.

**Also note:** `TRAIN_GROUP_COUNTS = [3498, 184, 56, 1057]` is asserted against the real data by a test. If that assert ever fails, group indexing is wrong and every number downstream is garbage. It's a tripwire.

### 4.3 `src/data.py` – loading images, fast

**What it does:** downloads Waterbirds, reads `metadata.csv`, exposes the three official splits, and builds the uint8 caches.

**Why the cache exists:** we evaluate the test set about 30 times during a sweep. Decoding 5,794 JPEGs each time would dominate everything. So we decode once, resize once, and keep raw pixels in a big tensor.




**The double-normalize trap**

Notice the cache stores **raw 0–255 pixels**, not normalized floats. That's deliberate and it's the single easiest way to wreck this project.
If you normalized before caching, and then the eval code normalized again (which it does, every time), every image would be silently wrong. Your accuracy would drop a bit, nothing would crash, and you'd spend a week blaming compression. The comment in the file calls this "the double-normalize trap". Caching pre-normalization is what avoids it.

**Look at:** `normalize_uint8()` – it's three lines and it's the only place normalization ever happens.

### 4.4 `src/model.py` – building the model and freezing the scope

**What it does:** creates the ViT, loads checkpoints, and defines exactly which weights compression may touch.

**Two functions that matter:**

- `targeted_named_weights(model)` – returns the list of **21,233,664** weights in scope. Every compression method starts by calling this. One definition, four methods.
- `fresh_model(ckpt, device)` – loads a brand new, clean, untouched model from the checkpoint file. The sweep calls this so that every configuration starts from the same pristine weights.

**Also:** `assert_structure()` checks the model really is 12 blocks × 6 heads × 64 dims, that `attn.norm` is an `Identity`, and that the matrices have the shapes we expect. This isn't paranoia – the head-removal maths in Section 5 is only correct *because* of that layout. If a future version of `timm` changes it, this assert fires instead of silently producing wrong results.

### 4.5 `src/evaluate.py` – one measurement pass

**What it does:** runs the model over a cached split and returns the full metric dict. It's 50 lines and you can read it in a minute.

**The one line to notice:** `model.eval()` at the top. Without it, dropout and drop-path stay active and inject random noise – which would look exactly like a compression effect. The comment says so.

**Also returns** `pred_mean`, the average predicted class. This is the collapse detector; see Section 5.

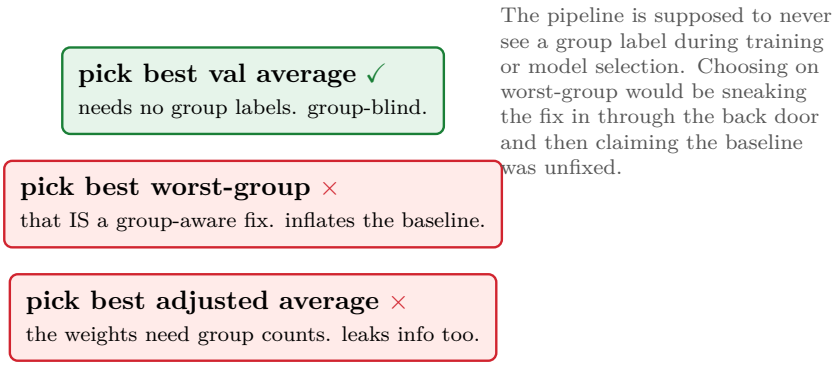
### 4.6 `src/train.py` – fine-tuning, deliberately naive

**What it does:** takes a model pretrained on ImageNet, swaps in a 2-class head, and fine-tunes on Waterbirds for 10 epochs. AdamW, learning rate 1e-5, cosine schedule, batch size 32.

**What it deliberately does NOT do:** no group labels, no reweighted loss, no balanced sampler, no GroupDRO, nothing clever. Plain shuffling, plain cross-entropy.

> **"Why would you train it badly on purpose?"**
> Because the model is *supposed* to learn the shortcut. That's the thing we want to study. If we trained it group-balanced, worst-group accuracy would start around 90%, there'd be no gap, and compression would have nothing interesting to break. The naive model is the subject of the experiment, not a mistake.

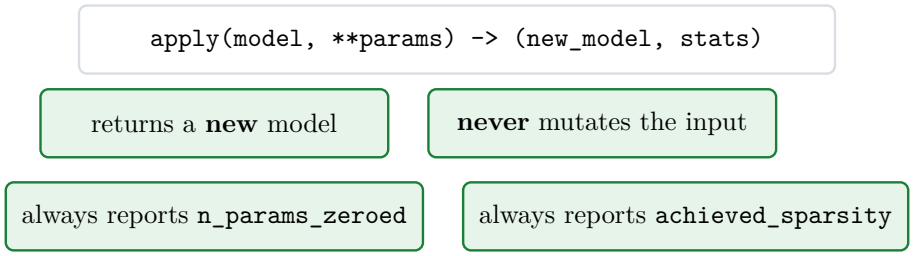
**The subtle choice – which epoch do we keep?** We save the checkpoint with the best **plain validation average** accuracy. Not worst-group. Here's why that matters:




**Phase 2 additions** (written, not run yet): `prune_indices` to train on a subset, `score_epoch` to record per-sample difficulty, and `stop_after_epoch` to score without overwriting an existing checkpoint. Section 6 explains what those are for.

### 4.7 `src/compress/` – the four methods, one contract

Every method, without exception, follows the same shape:




That contract is enforced by tests, not by good intentions. The shared helper `clone_model()` in `_common.py` is a deep copy – that's where "never mutates" starts.

**Masking, not shrinking**

Two of the methods (heads, ffn) delete whole structural chunks. You might expect them to physically make the tensors smaller. They don't – they set things to zero and leave the shapes alone.
Why: so `achieved_sparsity` means the same thing for all four methods. If head removal physically narrowed `qkv`, then "fraction of targeted weights zeroed" would be measuring something different than it does under magnitude pruning, and you couldn't put them on the same axis. The tests prove the masked version gives *bit-identical* outputs to really deleting the parts.

### `magnitude.py` – zero the smallest weights

Take all 21.2M weights, sort by absolute value, zero the smallest $k$ of them. Global ranking: a tiny weight in block 11 gets pruned before a big weight in block 0.

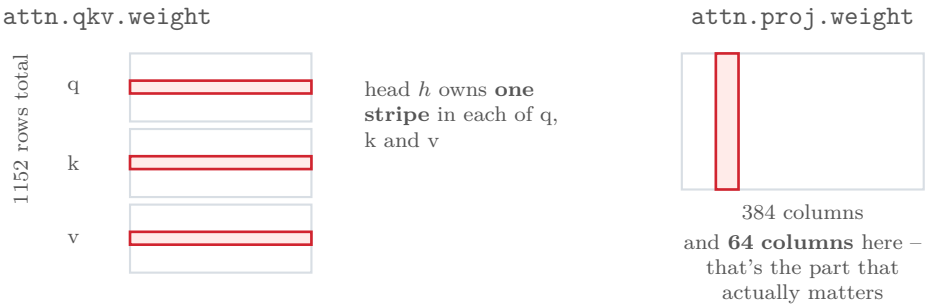



**The tricky bit – ties.** With 21 million float32 numbers, the threshold value is *not unique*. Several weights have exactly the same magnitude (measured: ties run 1–3 deep). If you wrote the obvious mask `w.abs() > threshold`, you'd zero *every* tied weight and overshoot your target. You'd ask for 30% and get 30.0001%.

So the code zeroes everything strictly below the threshold, counts how many that was, and then spends an explicit "tie budget" on exactly as many tied weights as it still needs – in a fixed order so it's reproducible. Result: exactly $k$ weights, every time. That's why the test is an equality and not a "close enough".

### `heads.py` – delete whole attention heads

Each block has 6 attention heads. A head owns a specific slice of two matrices:



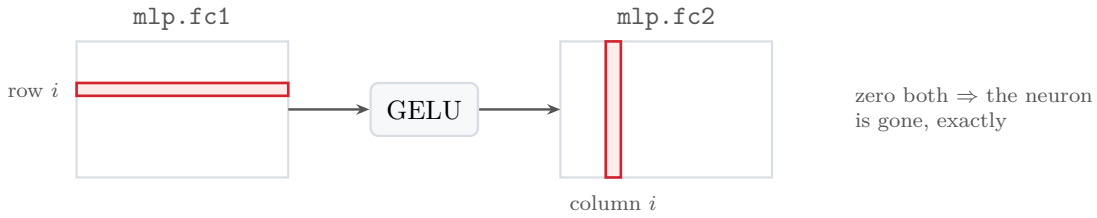

**Why zeroing those columns genuinely deletes the head.** At the end of attention, the 6 heads' outputs are laid side by side into one long vector, and then `proj` maps it back down. Head $h$'s output lands in columns $[h \cdot 64, (h{+}1) \cdot 64)$ of that vector. Zero those columns of `proj` and head $h$ contributes **exactly nothing** to the output. Not approximately – exactly.

The `qkv` rows get zeroed too. They can't change the output anymore (proj already blocks it), but they *are* dead weight, and leaving them out of the count would understate how much of the model a head sweep removes.

**One guard rail:** at least 1 head must survive per block. Empty a block's attention completely and it stops being a compressed transformer and becomes a different architecture – and the accuracy cliff would get blamed on "head sparsity". 12 blocks × 5 removable = 60 max, which is exactly where the sweep stops.

### `ffn.py` – delete whole MLP neurons

Simplest one conceptually. A "neuron" in the feed-forward layer is a matched pair: row $i$ of `fc1` (which computes it) and column $i$ of `fc2` (the only way it reaches the output).



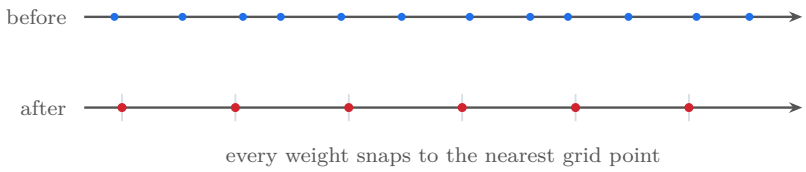

**The "wait, what about the bias?" question.** `fc1.bias` is *not* zeroed – biases are outside the frozen scope. So a pruned neuron still outputs a constant `GELU(bias_i)`. Doesn't that break exactness?

No. That constant gets multiplied by `fc2`'s zeroed column, so it contributes exactly zero. Removal stays exact without widening the scope. Nice little result, and there's a test that checks it against a reference model.

**No per-block floor here**, unlike heads. Emptying one block's MLP leaves attention and the residual stream intact – it degrades the model rather than changing its architecture. So blocks are free to give up wildly different numbers of neurons, and where the ranking chooses to cut is itself a result.

### `quantize.py` – fewer bits per weight

Instead of removing weights, store each one with less precision. Round every weight onto an evenly spaced grid.




**Two knobs, and one matters more than you'd think:**

- **bits** – 8, 6, 4, or 3. Fewer bits = coarser grid.
- **granularity** – `per_tensor` uses one grid spacing for the whole matrix; `per_channel` uses a separate one for each output row.

At 4 bits, per-channel keeps **60.6%** worst-group accuracy and per-tensor keeps **23.8%**. Same bit budget, wildly different outcome. One big outlier weight ruins the grid spacing for the entire tensor, and per-channel contains the damage to one row.

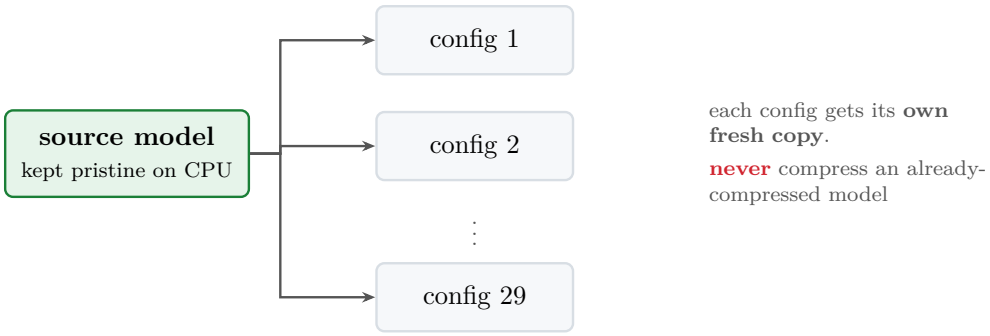
**Note:** this is *simulated* quantization – quantize then dequantize back to float32. We're measuring accuracy, not speed, and the numerical effect is reproduced exactly. Real int4 kernels are CUDA-only and wouldn't change a single accuracy number.

**Tiny detail worth knowing:** at `bits=32` the code short-circuits and returns the model untouched. If it actually ran the maths, dividing by a scale that small would push values past what float32's mantissa can hold, and the "identity" would be off by about 1e-7 – failing a test that demands exact identity.

### 4.8 `src/sweep.py` – the thing that actually runs the experiment

**What it does:** loops over all 29 configurations. For each one: load a fresh model, compress it, evaluate it, write a row to `results/sweep.csv`.

This is the most important file in the repo and it exists mostly to protect two invariants.




**The most dangerous bug available in this project**

If config 2 accidentally compressed the output of config 1, the damage would compound. And here's the evil part: **the output would still look completely plausible.** You'd get smooth, believable curves that are entirely wrong, and nothing would crash or look weird.

So the code doesn't just trust the control flow. Before every single config it recomputes a `_fingerprint()` of the source model – the count of zeros plus the sum of all weights – and asserts it hasn't changed. If any compression method ever broke its "don't mutate the input" promise, that assert fires immediately instead of 29 rows later.

**Invariant 2: resumability.** Each row is written and flushed to disk the moment it's done. A sweep is ∼30 full passes over 5,794 images; if your laptop sleeps halfway through, `-resume` picks up from the last finished row instead of starting over.

**Look at:** `run_one()` – it's about 25 lines and it's the entire experiment in miniature: compress, evaluate, build a row.

### 4.9 `src/figures.py` – turning the CSV into pictures

**What it does:** reads `sweep.csv` plus `baseline.json` and writes the three figures and the summary table.

**The one concept:** collapsed configurations are drawn as *hollow* markers and excluded from the trade-off plot. Section 5 explains why that matters so much.

### 4.10 Phase 2 files (written, not run yet)

`src/prune_data.py` – scores every training image by how "hard" it is, then removes the hardest ones in a class-balanced way and writes a list of which images to keep.

`src/retention.py` – compares the ERM model against the pruned model at matched compression and computes the retention ratio. This is Phase 2's headline number.

`src/migrate_csv.py` – a one-shot utility that adds the new `tag` column to the Phase 1 CSV so both phases share one schema. You'll never run it again.

`run_phase2.sh` – runs the whole Phase 2 pipeline in four stages. Every step skips if its output already exists, so an interrupted night resumes by just running it again.

### 4.11 `tests/` – the safety net

52 tests. They're not there for code coverage points; each one pins down a specific claim the rest of the code makes. The source comments literally refer to them by number ("V4 is an equality check").

| Gate | Claims | Why you'd care |
|---|---|---|
| V1 | group counts are `[3498,184,56,1057]` | group indexing is right |
| V3 | quantize at 32 bits is identity | the method has no hidden bias |
| V4 | magnitude zeroes *exactly* $k$ | the tie handling works |
| V5 | masking = real deletion, bit for bit | structured removal is exact |
| V7b | the baseline hasn't collapsed | accuracy isn't a mirage |
| V8 | `apply` never mutates its input | the sweep is trustworthy |
| V10 | pruned set is class-balanced | Phase 2 removal is fair |
| V12 | resume key separates seeds | a 2nd seed doesn't silently skip |

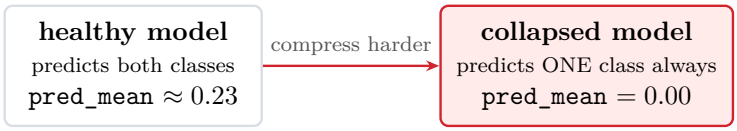
Run them with `pytest tests/ -q`. Takes about 35 seconds.

## 5. The tricky bits

This is the part that separates "I read the code" from "I understand the code". None of these are obvious, and most of them exist because someone would otherwise get a wrong answer that looks right.

### 5.1 Collapse: when the gap narrows and it means nothing

Look at the results past about 80% sparsity and something weird happens – the gap between average and worst-group accuracy starts *shrinking* again. Robustness returning? No. The opposite.




a model with one output has **no group disparity left to measure**. The gap looks small because the model is dead, not because it's fair.

Here's the real row from our data, at 80% magnitude sparsity:

| | g0 | g1 | g2 | g3 | adj avg |
|---|---|---|---|---|---|
| accuracy | 100% | 100% | 0% | 0% | **76.8%** |

That model predicts "landbird" for every single image. It gets 100% on both landbird groups and 0% on both waterbird groups. And it still posts a **76.8% adjusted average**, which would look like a mediocre-but-working model if you only read that one column.

That's why `pred_mean` exists in the CSV. It's the average predicted class; if it's pinned near 0 or 1, the model has collapsed. Those points get drawn hollow in the figures and excluded from the trade-off plot.

### 5.2 Head removal isn't monotonic, and that's fine

You'd expect: remove more heads, get worse. Our data says removing **48** heads scores *better* on worst-group (34.1%) than removing **36** (1.1%).

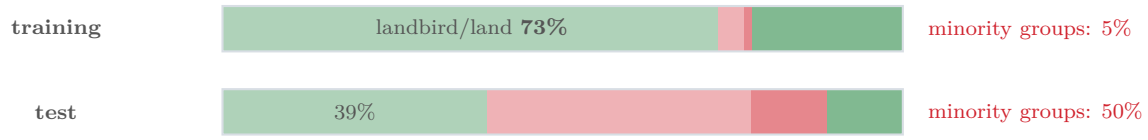
Two reasons, and neither is a bug:

- Ranking heads by weight size doesn't rank them by how much the model actually *uses* them. Cutting the "smallest" 36 can land worse than cutting the smallest 48.
- Near-collapse, which group is "worst" keeps reshuffling. At 36 heads the worst group is waterbird-on-land; the numbers are bouncing around a nearly dead model.

Treat the deep end of the head sweep as noise around a broken model, not as signal. The honest finding lives at the shallow end: 6 heads out of 72 costing 22 points.

### 5.3 Why the adjusted average, one more time

This trips everyone up, so here it is as a picture:

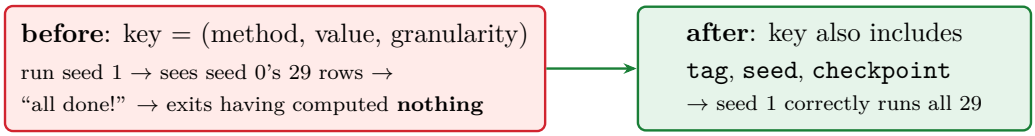



The test set is deliberately balanced; training isn't. A plain average over the test set therefore overweights rare groups relative to reality. The adjusted average re-weights back to training proportions, which is the convention in this research area. That's the only reason our headline is 98.2% instead of 91.7% – same model, same predictions, different weighting.

### 5.4 The resume bug (a real one we found and fixed)

Worth knowing because it's the kind of bug that produces *no error at all*.

The sweep skips configurations already in the CSV. The check used to be "have I seen this (method, value, granularity) before?". Which is fine when there's exactly one model.



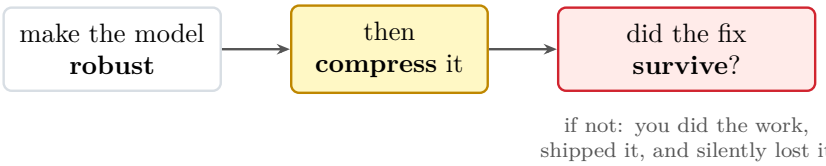

The CSV would have looked complete and you'd have "three seeds" that were actually one seed copied. Gate V12 now pins this down.

## 6. Phase 2: can we fix it, and does the fix survive?

**Status: code written, not run yet**

Everything in this part describes code that exists and is tested, but the experiment hasn't been executed. There are no Phase 2 numbers yet. Don't quote any.

Phase 1 compressed a model that was never made robust in the first place. Fair enough – but a practitioner would say "obviously, just fix the robustness first". Phase 2 does exactly that, and then asks the uncomfortable follow-up.




### 6.1 The fix: throw away the hardest training images

This is from a 2025 paper (Mulchandani and Kim, ICLR). The idea is counterintuitive and kind of beautiful:

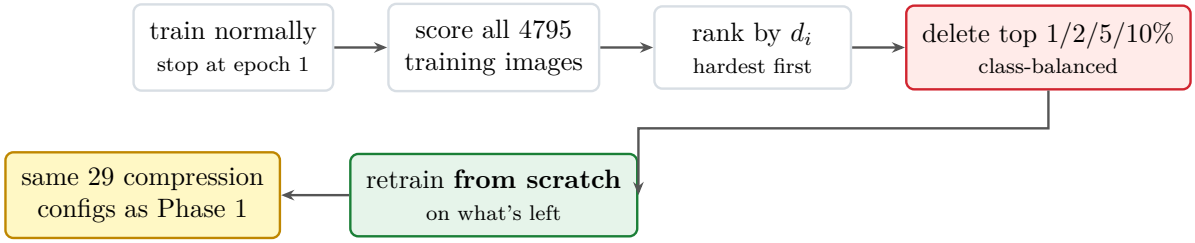
**Why deleting data makes a model MORE robust**

A model takes the background shortcut because some training images are hard to classify on their actual bird features. Where the real feature is hard, the easy fake feature wins.
So: find the images the model finds hardest, delete them, and retrain. Without those images pushing it toward the shortcut, the model has less reason to learn "wet background = waterbird". You lose a bit of data and gain robustness.

**How "hard" is measured.** Train normally, stop at an early epoch, and for each training image compute how far the model's predicted probabilities are from the correct answer:

$$d_i \;=\; \left\| \, p(x_i) - \text{onehot}(y_i) \, \right\|_2$$

Big $d_i$ means the model is confidently wrong or very unsure – a hard image.




**Two details that matter:**

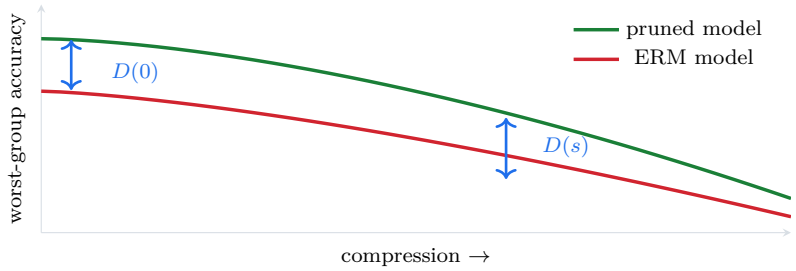
- **No group labels anywhere.** We never tell the pruner which images are minority-group. It only sees difficulty. That's essential – the whole point is a fix that works without knowing the groups, since in the real world you usually don't.
- **Class-balanced removal.** We delete the same *number* from each class, so the removal itself doesn't quietly rebalance the classes and create a different effect we'd mistake for robustness.

### 6.2 The measurement: retention ratio

Phase 2's headline number. Let $W_p$ and $W_e$ be worst-group accuracy of the **p**runed and **E**RM models at the same compression setting:

$$D(s) = W_p(s) - W_e(s) \qquad\qquad R(s) = \frac{D(s)}{D(0)}$$

In words: $D(0)$ is how much robustness the fix bought you before any compression. $D(s)$ is how much is left after compressing to level $s$. $R$ is the fraction that survived. In the sketch below, the left arrow is $D(0)$ – the gain the fix bought you – and the right arrow is $D(s)$, however much of it is still there after compressing.




$R \approx 1$ – the two curves stay apart.
your robustness fix survives deployment

$R \approx 0$ – the curves merge.
compression silently undid your fix

### 6.3 Two things the code refuses to do

- **It won't divide by noise.** If $D(0)$ is tiny – meaning the data pruning didn't actually help on our setup – then $R = D(s)/D(0)$ would produce enormous meaningless ratios out of rounding error. So there's a gate: if the dense gain is below a threshold, $R$ is left **undefined** and the result is reported as a null result. "It didn't work" is a real finding; a fabricated ratio is not.
- **It excludes collapsed models**, same as Phase 1. A dead model's "retention" is meaningless.

### 6.4 One non-obvious engineering bit

Scoring difficulty for seed 0 needs that seed's *epoch-1* weights. But seed 0 is already trained and its checkpoint is what produced all the Phase 1 results. Retraining it would overwrite that file – and training on Apple's GPU isn't bit-for-bit reproducible, so the new checkpoint wouldn't be identical. You'd silently decouple the published numbers from the weights behind them.

The fix is `-stop-after-epoch`: run the real schedule, stop early, write *no* checkpoint, keep only the scores. Costs 2 epochs instead of 10 and touches nothing.

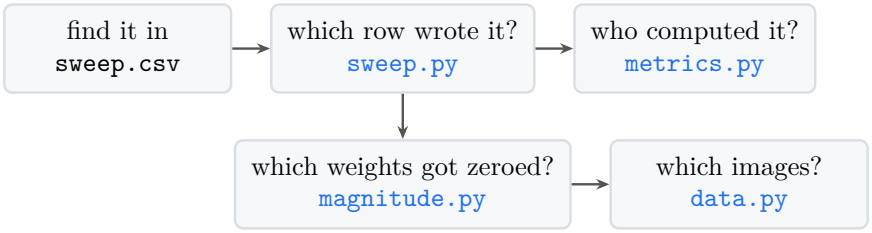
## 7. How to actually read this code

### 7.1 A reading order that works

1. `configs/base.yaml` – 2 minutes. Now you know every knob.
2. `src/metrics.py` – 10 minutes. Pure counting, no ML. Once `group_index` and `adjusted_average` click, the results table reads itself.
3. `src/evaluate.py` – 5 minutes. 50 lines. This is what "measuring the model" means here.
4. `src/compress/magnitude.py` – 15 minutes. The simplest method, and the tie-budget bit teaches you the mindset of the whole repo.
5. `src/sweep.py` – 20 minutes. Now you see how it all gets driven.
6. Everything else, as needed.

### 7.2 The exercise that makes it stick

Pick one number and chase it all the way through. Take **54.4%** – the worst-group accuracy after 30% magnitude pruning. Then:




Takes about half an hour and after it you'll know this codebase better than most people know code they wrote themselves.

### 7.3 Commands you'll actually use

| Command | What it does |
|---|---|
| `pytest tests/ -q` | run all 52 gates (∼35s) |
| `python -m src.figures` | rebuild the three figures from the CSV |
| `python -m src.sweep -seed 0` | re-run the sweep (resumes if interrupted) |
| `./run_phase2.sh` | the whole Phase 2 pipeline |
| `caffeinate -i <cmd>` | stop the Mac sleeping mid-run |

**One practical gotcha**

This machine is set to sleep after 1 minute idle. A sleeping Mac is why one training epoch in the Phase 1 log took **3338 seconds** when the others took **114**. Always prefix long runs with `caffeinate -i`, keep the lid open, and stay on mains power.

### 7.4 The three things to remember

1. **Average accuracy hides group damage.** 0.8 points vs 16 points, from the same operation.
2. **Structured compression hurts more than it looks.** 6 attention heads out of 72 cost 22 points of worst-group accuracy.
3. **A collapsed model can look fine.** 76.8% adjusted average from a model that predicts one class for every image. Always check `pred_mean`.

Built from the repo as it stands. Phase 1 results are real and reproducible; Phase 2 is code only, so far.